HTTP methods (GET, PUT, PATCH, POST, and DELETE, typically). The nouns are the resources in our application. We name those resources using URLs.

The Depot application that we produced contained a set of products. There are implicitly two resources here: first, the individual products, each of which constitutes a resource, and second, the collection of products.

To fetch a list of all the products, we could issue an HTTP GET request against this collection, say on the path /products. To fetch the contents of an individual resource, we have to identify it. The Rails way would be to give its primary key value (that is, its ID). Again we'd issue a GET request, this time against the URL /products/1.

To create a new product in our collection, we use an HTTP POST request directed at the /products path, with the post data containing the product to add. Yes, that's the same path we used to get a list of products. If you issue a GET to it, it responds with a list, and if you do a POST to it, it adds a new product to the collection.

Take this a step further. We've already seen you can retrieve the content of a product—you just issue a GET request against the path /products/1. To update that product, you'd issue an HTTP PUT request against the same URL. And, to delete it, you could issue an HTTP DELETE request, using the same URL.

Take this further. Maybe our system also tracks users. Again, we have a set of resources to deal with. REST tells us to use the same set of verbs (GET, POST, PATCH, PUT, and DELETE) against a similar-looking set of URLs (/users, /users/1, and so on).

Now we see some of the power of the constraints imposed by REST. We're already familiar with the way Rails constrains us to structure our applications a certain way. Now the REST philosophy tells us to structure the interface to our applications too. Suddenly our world gets a lot simpler.